Bird Wanderer Test Cases

Five successful operations and five unsuccessful operations with error handling

Purpose: Demonstrate the main project workflows and show how the application rejects invalid operations, protects stored information and recovers from errors.

Format: Lecture 11, slide 4: case identification, priority, module, dates, preconditions, dependencies, step table, results, notes and postconditions.

Execution record: 26 October 2026. Ten automated API integration cases passed against the current application routes, SQLite and image processing. External eBird responses and the photograph use controlled synthetic fixtures. Browser steps below are rehearsal instructions, not recorded screen execution.

Demonstration: 27 October 2026. In a negative case, the requested operation is unsuccessful, but the test status is Pass when the expected rejection and error handling occur. Fail means the application behaved differently from the expected result.

Case	Successful operation	Case	Unsuccessful operation and handling
BW-P01	Discovery and catalogue integration	BW-N01	Corrupt photograph handling
BW-P02	Photograph upload and EXIF privacy	BW-N02	Upload size limit and recovery
BW-P03	Private observation and journal	BW-N03	Observation date validation
BW-P04	Public sharing with location protection	BW-N04	Private observation access control
BW-P05	Bookmarks and account isolation	BW-N05	External discovery failure and recovery

Coverage: External discovery; image processing; EXIF privacy; observation creation; journal/profile counts; community visibility; bookmarks; validation; access control; provider recovery.

How to use: Each following case sheet has the lecture fields and verified API results. Use the screen demonstration paragraph to rehearse the same behavior. The last page provides the presentation order and a repeatable error demonstration.

BW-P01 Discovery and catalogue integration

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-P01	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Discovery and catalogue integration	Test Execution date	26 October 2026
Test Title	Discover species, hotspots and recent sightings	Test Case Status	Pass at API level

Description: Load catalogue species, hotspots and recent sightings through the discovery service.

Pre-conditions: Owner account is signed in. Controlled provider is available. Catalogue cache TTL is zero for this demonstration.

Dependencies: Authentication session; discovery service; controlled eBird taxonomy, hotspot and report responses.

Test Data: Indian Roller (indrol2); Demo Wetland (L123); one recent report; 42 provider-reported species at the hotspot.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	GET /api/discovery/catalogue	200 OK. Keep provider identifiers and join the recent report to the correct hotspot. Do not expose the server API key.	200 OK; Indian Roller joins to Demo Wetland and its recent report. Source IDs retained; API key absent.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Species indrol2 and hotspot L123 are linked correctly; hotspot speciesCount is 42; key is absent from the response.	Species indrol2 and hotspot L123 are linked correctly; hotspot speciesCount is 42; key is absent from the response.	Pass	Assertions passed in the demonstration runner.

Post-conditions: Catalogue is available to Explore/Search. No observation has been created.

Screen demonstration: Open Explore or Search after signing in. Open a catalogue bird and its hotspot. Explain where species names, hotspot identifiers and recent sightings come from.

Project explanation: This demonstrates the external-data path: screen request, protected server route, provider response, normalized catalogue and visible discovery results.

BW-P02 Photograph upload and EXIF privacy

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-P02	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Photograph upload and EXIF privacy	Test Execution date	26 October 2026
Test Title	Upload a valid photograph and protect its metadata	Test Case Status	Pass at API level

Description: Accept a valid photograph, extract available camera metadata and remove embedded EXIF from the served image.

Pre-conditions: Owner is signed in. A valid JPEG fixture with camera metadata and synthetic GPS coordinates is available.

Dependencies: Upload route; image decoder and re-encoder; EXIF parser; media directory; SQLite media table.

Test Data: Valid 16 x 16 JPEG test fixture; camera make Demo camera; synthetic GPS 28.5, 77.2; Content-Type image/jpeg.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	POST /api/observations/media; then GET /api/observations/media/{mediaId}	201 Created for upload and 200 for owner image access. Return extracted metadata, flag private GPS and serve a JPEG without embedded EXIF.	201 Created; camera metadata extracted; private GPS detected; served JPEG has no embedded EXIF; one media record.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Camera make is Demo camera; hasPrivateLocation is true; served image EXIF is absent; exactly one media row exists.	Camera make is Demo camera; hasPrivateLocation is true; served image EXIF is absent; exactly one media row exists.	Pass	Assertions passed in the demonstration runner.

Post-conditions: One staged photograph belongs to the owner. Upload alone does not create a journal observation.

Screen demonstration: Open Log Observation and select a valid JPEG/PNG. Show the photo preview and real metadata fields. Explain that missing EXIF stays unavailable and that the server re-encodes the shared photo.

Project explanation: This explains file validation, camera metadata, media storage and why exact GPS from the original file must not travel inside a shared image.

BW-P03 Private observation and journal

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-P03	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Private observation and journal	Test Execution date	26 October 2026
Test Title	Save a private observation to the journal	Test Case Status	Pass at API level

Description: Create a private bird observation and verify its journal entry and derived profile counts.

Pre-conditions: Owner is signed in. BW-P02 supplies an owned, unused photo. The catalogue contains the selected bird and hotspot.

Dependencies: Owned media; catalogue species/hotspot; observation transaction; journal queries; profile aggregates.

Test Data: Uploaded mediaId; indrol2; L123; 2026-10-25T09:00; private; hideExactLocation=true; demonstration field notes.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	POST /api/observations; GET /api/observations; GET /api/observations?scope=feed as Viewer	201 Created. Owner journal includes the observation and counts one photo and one bird. Viewer community feed excludes the private record.	201 Created; owner journal contains one observation; photo and bird totals are 1; other account feed excludes it.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Journal contains one record; photosCount=1 and birdsCount=1; viewer feed contains zero records; owner can read private GPS.	Journal contains one record; photosCount=1 and birdsCount=1; viewer feed contains zero records; owner can read private GPS.	Pass	Assertions passed in the demonstration runner.

Post-conditions: One private observation links the owner, photograph, species, hotspot and observation date.

Screen demonstration: Complete Log Observation with a catalogue bird, hotspot, past date, notes and Private visibility. Save it, then open Journal and Profile to show the resulting record and counts.

Project explanation: This connects the main workflow from photograph and field details to a database record, journal display and profile statistics.

BW-P04 Public sharing with location protection

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-P04	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Public sharing with location protection	Test Execution date	26 October 2026
Test Title	Share an observation while withholding exact coordinates	Test Case Status	Pass at API level

Description: Allow another signed-in account to read a public observation while withholding exact private coordinates.

Pre-conditions: BW-P03 created a private observation. Owner and Viewer use separate authenticated sessions. Owner profile permits public sharing.

Dependencies: Observation visibility update; server read policy; response serialization; protected photo route; feed query.

Test Data: visibility=public; same record and media from BW-P03; synthetic exact coordinates remain 28.5, 77.2.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	PATCH /api/observations/{observationId}; then GET the record, photo and community feed as Viewer	200 OK. Viewer sees the public post, its photo and hotspot name. The shared record omits privateLocation.	200 OK; other account can read the public post and photo; hotspot name shown; privateLocation absent.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Viewer reads post and photo with 200; feed includes the post; location is Demo Wetland; privateLocation property is absent.	Viewer reads post and photo with 200; feed includes the post; location is Demo Wetland; privateLocation property is absent.	Pass	Assertions passed in the demonstration runner.

Post-conditions: The observation is public. Shared location remains at hotspot level and the served photo contains no EXIF.

Screen demonstration: Using the owner account, make a record Public. Open Community in a second browser profile as Viewer. Show the post and hotspot-level location, then explain the separate private-coordinate field.

Project explanation: This demonstrates that publishing a sighting does not require publishing its exact photo coordinates. The server controls the shared response.

BW-P05 Bookmarks and account isolation

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-P05	Test Designed date	26 October 2026
Test Priority	Medium	Test Executed by	Codex automated API runner
Module Name	Bookmarks and account isolation	Test Execution date	26 October 2026
Test Title	Bookmark a catalogue species without duplicate records	Test Case Status	Pass at API level

Description: Save a catalogue species, avoid duplicate bookmarks and keep the saved list specific to its owner.

Pre-conditions: Owner and Viewer are signed in separately. indrol2 is in the controlled catalogue. Neither account has bookmarks.

Dependencies: Activity route; trusted catalogue validation; SQLite unique key; account-specific activity query.

Test Data: Both save requests use {saved:true}. Species identifier indrol2 belongs to the current catalogue.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	PUT /api/activity/saves/species/indrol2 twice; then GET /api/activity for both accounts	200 OK. Repeating the same save creates one reference. A later read returns it for Owner and no bookmark for Viewer.	200 OK; two save requests produce one saved reference; later read retains it; viewer has no bookmark.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Exactly one discovery_saves row; owner read returns indrol2; viewer saved list is empty.	Exactly one discovery_saves row; owner read returns indrol2; viewer saved list is empty.	Pass	Assertions passed in the demonstration runner.

Post-conditions: Owner has one saved species reference. Viewer account remains unchanged.

Screen demonstration: Open a catalogue species and click Bookmark species. Reload and confirm it stays saved. Use another account to show its separate saved list. The automated case also checks duplicate save requests directly.

Project explanation: This explains persistent preferences, the relationship between bookmarks and catalogue IDs, and account separation.

BW-N01 Corrupt photograph handling

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-N01	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Corrupt photograph handling	Test Execution date	26 October 2026
Test Title	Reject a corrupt photograph and allow a valid replacement	Test Case Status	Pass at API level

Description: Reject a file that looks like a JPEG at the header level but cannot be decoded, then accept a valid replacement.

Pre-conditions: Owner is signed in. Existing media and file totals are recorded. No upload is currently running.

Dependencies: Image signature check; image decoder; error propagation; media database and file storage.

Test Data: Corrupt bytes FF D8 FF 00; Content-Type image/jpeg. Recovery uses the valid JPEG fixture.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	POST /api/observations/media with corrupt bytes; then retry with a valid JPEG	422 Unprocessable Entity. Show the decode error, create no media row/file for the bad upload, and accept a valid replacement with 201.	422 Unprocessable Entity; corrupt JPEG creates no media row or file; valid replacement returns 201.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Media and file totals remain unchanged after rejection; the replacement upload returns 201.	Media and file totals remain unchanged after rejection; the replacement upload returns 201.	Pass	Assertions passed in the demonstration runner.

Error message: Cannot decode this photograph. Use a valid single-frame JPEG/PNG of at most 40 megapixels.

Recovery: Choose a valid JPEG/PNG. A valid retry succeeded.

Post-conditions: Bad content has not become a media record. A valid replacement is staged successfully.

Screen demonstration: In Log Observation select a corrupt .jpg file. Show the decode error and that publishing cannot proceed with that upload. Choose a valid photo and show successful recovery.

Project explanation: This shows why an extension or MIME type is insufficient. Bird Wanderer also decodes the image before committing it.

BW-N02 Upload size limit and recovery

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-N02	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Upload size limit and recovery	Test Execution date	26 October 2026
Test Title	Reject an upload above the 10 MB limit	Test Case Status	Pass at API level

Description: Reject a photograph upload over 10 MB without adding stored files, then accept a smaller valid photograph.

Pre-conditions: Owner is signed in. Record media row and file totals before sending the oversized request.

Dependencies: Browser file-size guard; server raw-body limit; shared API error handler; media storage.

Test Data: 10 * 1024 * 1024 + 1 bytes; image/jpeg. This exceeds the 10,485,760-byte application limit.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	POST /api/observations/media with 10,485,761 bytes; then retry with a small valid JPEG	API returns 413 with Request is too large. No new media row or file. A smaller valid retry returns 201.	413 Payload Too Large; 10,485,761-byte upload rejected; file/row totals unchanged; smaller valid retry returns 201.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Oversized upload returns 413; file and row totals are unchanged; smaller valid upload returns 201.	Oversized upload returns 413; file and row totals are unchanged; smaller valid upload returns 201.	Pass	Assertions passed in the demonstration runner.

Error message: Request is too large.

Recovery: Resize/compress the photo below the limit. A smaller valid upload succeeded.

Post-conditions: No oversized media persists. The server remains available for a valid retry.

Screen demonstration: Select a JPEG/PNG larger than 10 MB in Log Observation. The browser should show Choose a JPEG/PNG photograph up to 10 MB. Use the runner to show the independent server-side 413 guard, then retry with a small photo.

Project explanation: This explains two layers of validation: the browser gives early feedback and the server enforces the limit even if a client bypasses the screen.

BW-N03 Observation date validation

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-N03	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Observation date validation	Test Execution date	26 October 2026
Test Title	Reject an impossible future observation date	Test Case Status	Pass at API level

Description: Reject an implausibly future observation date and allow correction without re-uploading the staged photograph.

Pre-conditions: Owner is signed in. A fresh unused photo is staged. Species, hotspot, visibility and notes are otherwise valid.

Dependencies: Observation form; date parser and range check; owned staged photo; observation insert transaction.

Test Data: observedAt=2099-01-01T09:00. Recovery: 2026-10-25T09:00 using the same mediaId.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	POST /api/observations with a future date; repeat with only the date corrected	400 Bad Request with the observation date/time error. No observation added. Correcting the date returns 201 with the same staged photo.	400 Bad Request; 2099 date creates no observation; correcting only the date returns 201 with the same staged photo.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Observation count stays unchanged on failure; corrected submission creates the record successfully.	Observation count stays unchanged on failure; corrected submission creates the record successfully.	Pass	Assertions passed in the demonstration runner.

Error message: Enter a valid observation date/time (camera/local clock; timezone unverified).

Recovery: Correct the date to the actual observation time. Corrected submission succeeded without re-upload.

Post-conditions: Only the corrected observation exists. The failed attempt did not consume the staged photo.

Screen demonstration: Fill all Log Observation fields correctly, but enter 1 January 2099. Submit and show the date error. Correct it to a genuine past observation time and save again without choosing another photo.

Project explanation: This demonstrates domain validation, preservation of recoverable work and protection of the journal against impossible dates. Camera timezone remains unverified.

BW-N04 Private observation access control

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-N04	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	Private observation access control	Test Execution date	26 October 2026
Test Title	Block another account from a private record and its photograph	Test Case Status	Pass at API level

Description: Prevent another signed-in account from reading or deleting a private observation or fetching its photo.

Pre-conditions: The owner changes the record from BW-P03 back to Private. Viewer is a different authenticated account.

Dependencies: Session ownership; server visibility policy; observation routes; protected media route.

Test Data: Use the real record and media identifiers from Owner, but send Viewer session credentials.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	As Viewer: GET and DELETE /api/observations/{id}, and GET /api/observations/media/{mediaId}	404 Observation not found for record access/delete and 404 Photo not found for image access. Owner still reads the record with 200.	404 for private record, photo and delete attempt from viewer; owner still reads the unchanged record with 200.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	Viewer read, photo request and delete all return 404; owner still reads the unchanged observation with 200.	Viewer read, photo request and delete all return 404; owner still reads the unchanged observation with 200.	Pass	Assertions passed in the demonstration runner.

Error message: Observation not found.

Recovery: Use the owning account or ask the owner to change visibility. Owner access succeeded.

Post-conditions: The private record and media remain intact and available to their owner. Viewer receives no private record.

Screen demonstration: Keep the owner record Private. In a separate browser profile sign in as Viewer and open the shared observation link. Show denied access. Use the runner for direct photo/delete attempts and switch back to Owner to prove availability.

Project explanation: This explains authorization after login. A valid account still cannot access every record. Returning 404 also avoids confirming a private record exists.

BW-N05 External discovery failure and recovery

Field	Value	Field	Value
Project Name	Bird Wanderer	Test Designed by	Project team with Codex assistance
Test Case ID	BW-N05	Test Designed date	26 October 2026
Test Priority	High	Test Executed by	Codex automated API runner
Module Name	External discovery failure and recovery	Test Execution date	26 October 2026
Test Title	Handle a bird-data provider outage and recover on retry	Test Case Status	Pass at API level

Description: Handle loss of the eBird connection explicitly while keeping locally stored journal records available.

Pre-conditions: Owner is signed in. Journal records already exist. Provider failure is controlled and the catalogue cache is disabled in the runner.

Dependencies: Discovery provider client; exception mapping; catalogue endpoint; independent local journal storage.

Test Data: Controlled provider fetch throws a network error. Recovery restores the same synthetic provider responses.

Step	Test Steps	Test Data	Expected Result	Actual Result	Status (Pass/Fail)	Notes
1	Execute the operation through the application API.	GET /api/discovery/catalogue during provider failure; GET journal; restore provider and retry catalogue	503 Service Unavailable with Cannot reach eBird. Please try again. Do not return fake empty species. Journal read returns 200 and catalogue retry returns 200.	503 Service Unavailable; explicit error instead of fake empty data; journal remains readable; provider recovery returns 200.	Pass	Controlled local API execution.
2	Verify stored data, access policy and the resulting state.	Inspect returned records and relevant SQLite rows/files.	503 error response contains no species list; journal remains readable with 200; restored provider returns indrol2 with 200.	503 error response contains no species list; journal remains readable with 200; restored provider returns indrol2 with 200.	Pass	Assertions passed in the demonstration runner.

Error message: Cannot reach eBird. Please try again.

Recovery: Restore provider connectivity and retry. Catalogue retry succeeded; local journal remained available.

Post-conditions: No journal data is lost. Discovery resumes after connectivity returns.

Screen demonstration: Use the runner for a repeatable provider-only outage. Explain the Explore/Search Try again control. For a screen rehearsal, block only the discovery request in browser developer tools; unblock and retry. Browser blocking may show the client network error instead of the server 503 text.

Project explanation: This separates external discovery availability from locally stored field records. An outage must be reported as an error rather than as no birds found.

Demonstration Guide

Opening statement: Bird Wanderer helps users discover birds and hotspots, upload photographs and keep field observations. These cases follow that workflow and then test what happens when files, dates, access rights or external services are invalid.

Suggested order: P01 discovery, P02 photo upload, N01 corrupt photo, N02 oversized photo, N03 future date, P03 private journal, N04 private access, P04 public sharing, P05 bookmarks, N05 provider failure. Reset or use separate records when switching visibility.

Before class: Start the app using the existing project launcher Test.ps1. Prepare two separate browser profiles as Owner and Viewer. Use a real valid JPEG/PNG for the screen workflow, a corrupt .jpg and a JPEG/PNG over 10 MB. Ensure the selected species/hotspot is present in the live catalogue. Demo Wetland is a test fixture, so use a real loaded hotspot in the app.

Repeatable demonstration: From a PowerShell terminal in the project folder, run: .\docs\test-cases\Run-Demo.ps1. It prints all ten results, the exact error messages and the recovery checks. It uses temporary accounts, a local SQLite database and a synthetic photo/provider. It does not read the project .env or modify real account data.

Important screen distinction: The oversized file normally fails in the browser first with Choose a JPEG/PNG photograph up to 10 MB. The runner bypasses that guard to verify the server returns 413 and Request is too large. Corrupt JPEG decoding returns 422. Describe both layers accurately.

Privacy demonstration: Private access returns 404 rather than revealing whether a protected record exists. Test the record and its photo, then show that Owner can still read it. For the public case, a hotspot name may remain visible while exact EXIF coordinates stay private.

Provider demonstration: Use the runner for a controlled eBird outage and recovery without depending on tomorrow's network. It executes the real provider error-mapping path with an injected connection failure. Live provider availability, screen rendering and physical-device behavior are separate acceptance checks.

If asked why negative tests pass: The operation failed as intended. The test passed because the application showed the expected error, preserved the correct data and allowed a safe recovery.

Result record: All 10 API cases passed on 26 October 2026. Screen execution by presenter: _____ Date: _____. Record any mismatch as Fail and note what actually appeared.

References: lecture11x_SE.pptx, slide 4; frontend/server/discovery.mjs; frontend/server/observations.mjs; frontend/server/activity.mjs; frontend/server/auth.mjs; frontend/src/components/LogObservationScreen.tsx; frontend/src/App.tsx. Executable evidence: docs/test-cases/run-demo.mjs.